We could use Repo.update_all/3[18] to directly send an UPDATE statement to the database. update_all is useful in situations where you want to update in a single operation, but it's a best practice to update with a changeset. This forces your application to verify that the user has access to the record being updated, and you'll receive a copy of the updated struct from the Repo.update/1 function.

We have the persistence functions we need for SMS messages, so let's move on to querying for data. We'll be using Ecto's query builder DSL for this.

Association Functions



The Ecto.Changeset module exposes put_assoc/4 and cast_assoc/3 for interacting with associations, but they are optional. When you use these functions, Ecto detects changes in the association data and will insert or update those database records.

I find that casting associations as regular fields—like we did with contact_id—removes a layer of magic and is much simpler to understand. So I don't use the association changeset functions.

Use Ecto.Query for More Advanced Queries

Ecto.Query[19] provides a query builder DSL that gives you complete control over your database queries. Ecto queries end up feeling like SQL, but they have extra protection that comes from Elixir's compile-time guarantees.

Ecto knows about our application schemas because they are defined in code. It uses this knowledge to verify that we're querying columns that exist and that they are aliased correctly. Ecto uses macros and compile-time checks to guarantee that any data sent to the database has been properly sanitized. In the end, we get a familiar syntax with extra safety!

Let's ease into Ecto.Query with a simple function that loads all messages with a given contact. Add this to the bottom of SmsMessageStore:

18. https://hexdocs.pm/ecto/Ecto.Repo.html#c:update_all/3
19. https://hexdocs.pm/ecto/Ecto.Query.html